CGE-P LAB NOTES

Lab 4.3: Building a GRC Evidence Pipeline (AWS + GitHub Actions)

The local Conftest gate from Lab 3.4 catches violations on your laptop. CI catches them across the whole team. This lab wires the gate into GitHub Actions, runs it on every pull request, and uploads a named evidence artifact for every run.

The YAML file you commit **is** your CM-3, CM-6, CA-2, CA-7, RA-5, SR-3, and AU-9 evidence.

For reading. Code lines wrap to fit the page; copy code from docs/04_03_grc_evidence_pipeline.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/04_03_grc_evidence_pipeline.md` in your clone, not from the PDF.

Learning objectives

- Wire AWS OIDC trust so the workflow assumes a role with no long-lived keys.
- Scope that role to exactly what it needs, including the Lab 2.2 state backend.
- Run plan, Conftest, and a vulnerability scan on every PR, failing closed.
- Pin the supply chain: actions by commit SHA, tools by version and checksum.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 2.2** state backend. The pipeline needs it; see "Why the backend is a prerequisite" below.
- Lab 2.3, Lab 3.3, Lab 3.4 artifacts committed into the repo.
- A GitHub repository you own.
- AWS permissions to create an IAM OIDC provider and role.

Estimated time & cost

- 90 minutes.
- Free. GitHub Actions' free tier covers this, and the workflow only plans.

Why the backend is a prerequisite

Run `terraform init` on a fresh runner with a local backend and the runner has no state, so `plan` reports every resource as "to be created."

For a plan-only workflow that is merely misleading. For the capstone, which requires `Apply` on merge to `main`, it is fatal: the first apply collides with your existing resources or silently builds a second copy of everything.

Lab 2.2 exists to prevent this. If you skipped it, stop and do it now, because every diff this pipeline produces is fiction until you have.

Architecture

```
  PR opened
      |
      v
  workflow run
      |-- Configure AWS creds (OIDC, no keys on disk)          IA-5
      |-- terraform init  (reads Lab 2.2 remote state)
      |-- terraform fmt -check                                 CM-6
      |-- terraform plan  (a REAL diff, because state exists)
      |-- Conftest gate via scripts/policy-gate.sh             CA-2, fail closed
      |-- Trivy config scan                                    RA-5, fail closed
      |-- Upload evidence artifact                             AU-9
      +-- Comment on PR with the summary
```

Lab 4.4 adds Cosign signing and uploads the bundle to the Lab 2.5 vault.

Step-by-step walkthrough

Step 1 OIDC trust, scoped properly

```
# oidc/main.tf
terraform {
  required_version = ">= 1.10"

  backend "s3" {
    bucket       = "cgep-lab-tfstate-XXXXXXXX"
    key          = "labs/4-3-oidc/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }

  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region = "us-east-1"
  default_tags {
    tags = {
      Project         = "cgep-lab"
      Environment     = "shared"
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
```

```
}

variable "github_org"    { type = string }
variable "github_repo"   { type = string }
variable "state_bucket"  { type = string }
variable "state_kms_arn" { type = string }

data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

# AWS now maintains the trust chain for this provider internally, so the
# thumbprint is no longer load-bearing. It remains a required argument.
resource "aws_iam_openid_connect_provider" "github" {
  url             = "https://token.actions.githubusercontent.com"
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

resource "aws_iam_role" "grc_gate" {
  name = "cgep-grc-gate"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Federated = aws_iam_openid_connect_provider.github.arn }
      Action    = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          "token.actions.githubusercontent.com:sub" = "repo:${var.github_org}/${var.github_repo}:*"
        }
      }
    }]
  })
}

# AC-6: read-only for the plan itself.
resource "aws_iam_role_policy_attachment" "readonly" {
  role       = aws_iam_role.grc_gate.name
  policy_arn = "arn:${data.aws_partition.current.partition}:iam::aws:policy/ReadOnlyAccess"
}

# The state backend needs more than read. `terraform plan` writes a lock
# object and refreshes state. Without this the job fails at init.
resource "aws_iam_role_policy" "state_access" {
  name = "tfstate-access"
  role = aws_iam_role.grc_gate.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect   = "Allow"
```

```
        Action   = ["s3:ListBucket", "s3:GetBucketLocation"]
        Resource = "arn:${data.aws_partition.current.partition}:s3:::${var.state_bucket}"
      },
      {
        Effect = "Allow"
        Action = ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"]
        Resource = "arn:${data.aws_partition.current.partition}:s3:::${var.state_bucket}/*"
      },
      {
        Effect   = "Allow"
        Action   = ["kms:Decrypt", "kms:GenerateDataKey"]
        Resource = var.state_kms_arn
      },
    ]
  })
}

output "role_arn" { value = aws_iam_role.grc_gate.arn }
```

```
cd oidc
terraform init
terraform apply
```

The `StringLike` on `sub` binds this role to one repository. **Do not loosen it.** A role trusted by `repo:*:*` is trusted by every public repository on GitHub, which is not a subtle failure. If you want it tighter still, bind to a specific ref or environment:

```
"repo:YourOrg/YourRepo:ref:refs/heads/main"
"repo:YourOrg/YourRepo:environment:production"
```

The environment form is what you want for the capstone's apply job, because it lets a GitHub environment protection rule gate the credential itself.

If the OIDC provider already exists in the account:

```
terraform import aws_iam_openid_connect_provider.github \
  arn:aws:iam::ACCOUNT:oidc-provider/token.actions.githubusercontent.com
```

Step 2 Repository variables

`cgep.env` holds these values on your machine, and CI cannot read that file. It needs its own copy. Derive them from the environment rather than retyping, so the two cannot drift apart:

```
# You are in oidc/ after Step 1, so cgep.env is one level up, not two.
source ../cgep.env
gh variable set AWS_ROLE_ARN        --body "arn:aws:iam::$(aws sts get-caller-identity --query
Account --output text):role/cgep-grc-gate"
gh variable set TF_STATE_BUCKET     --body "$TF_VAR_state_bucket"
gh variable set TF_VAR_PROJECT_NAME --body "$TF_VAR_project_name"
gh variable set TF_VAR_ENVIRONMENT  --body "$TF_VAR_environment"
```

Run that from inside your repository and `gh` targets it automatically; add `--repo OWNER/NAME` only if you are somewhere else.

`TF_VAR_PROJECT_NAME` and `TF_VAR_ENVIRONMENT` are the workspace's only two required inputs. Locally they come from `cgep.env`; CI cannot read that file. They also form your bucket names, which makes a wrong value worse than a missing one: it does not error, it proposes renaming infrastructure you already have. The workflow refuses to run if either arrives empty, because an unset repository variable renders as an empty string rather than as nothing.

`TF_STATE_BUCKET` is not decoration. Your backend is partial: `main.tf` carries the state key, and `backend.hcl` carries the bucket, and `backend.hcl` is gitignored because it names your account's resources. So CI has no `backend.hcl` at all, and a bare `terraform init -input=false` fails rather than prompting. The workflow passes the bucket with `-backend-config` from this variable, which is the whole reason it exists.

Variables, not secrets, and the distinction is the lesson. A role ARN and a bucket name are configuration. They identify things; they do not grant access to them. GitHub **secrets** are write-only and masked in logs, which is right for a credential and actively unhelpful for a value you want to read in a failed run. Putting non-secrets in `secrets` is a common habit and it quietly trains people that everything is equally sensitive, which is how real credentials end up treated casually.

Notice what is **not** here: no access key, no secret key, no long-lived credential of any kind. That is the point of the OIDC trust you just built. The role ARN being public costs you nothing, because assuming it requires a token that only your repository, on your ref, can obtain. If this lab had you paste an access key into `secrets`, it would be teaching the opposite of what it claims to teach.

Step 3 The workflow

```
# .github/workflows/grc-gate.yml
# Lab 4.3: the GRC evidence pipeline.
# Every line here is a control statement; see the guide's takeaway table.
name: grc-gate

on:
  pull_request:
    branches: [main]
  workflow_dispatch:

# AC-6: the minimum token scope that works.
```

```
permissions:
  id-token: write      # AWS OIDC, and Cosign keyless in Lab 4.4
  contents: read
  pull-requests: write # PR comment

env:
  AWS_REGION: us-east-1
  TF_WORKING_DIR: terraform/primitives/compliant-s3
  TERRAFORM_VERSION: 1.15.8
  CONFTEST_VERSION: 0.69.0
  OPA_VERSION: 1.19.1
  TRIVY_VERSION: 0.74.0

  # The workspace's two required inputs. Locally they come from cgep.env, which
  # CI cannot read, so they arrive as repository variables. They form the bucket
  # names, so they must match what built the resources: a wrong value does not
  # error, it proposes renaming live infrastructure.
  TF_VAR_project_name: ${{ vars.TF_VAR_PROJECT_NAME }}
  TF_VAR_environment: ${{ vars.TF_VAR_ENVIRONMENT }}

jobs:
  grc-gate:
    runs-on: ubuntu-latest
    steps:
      # SR-3 / SR-11: actions pinned by commit SHA, not by tag.
      # A tag is a mutable pointer. Pinning to v4 means "whatever the
      # maintainer, or whoever compromises their account, decides v4 means."
      - name: Checkout
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@e6de054238d6b7531b4efff3b6587d9aade6a06c # v6.2.3
        with:
          role-to-assume: ${{ vars.AWS_ROLE_ARN }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@dfe3c3f87815947d99a8997f908cb6525fc44e9e # v4.0.1
        with:
          terraform_version: ${{ env.TERRAFORM_VERSION }}
          terraform_wrapper: false

      - name: Install Conftest and OPA
        run: |
          set -euo pipefail
          curl -fsSL "https://github.com/open-policy-agent/conftest/releases/download/v$
{CONFTEST_VERSION}/conftest_${CONFTEST_VERSION}_Linux_x86_64.tar.gz" \
            | tar -xz -C /usr/local/bin conftest
          curl -fsSL "https://github.com/open-policy-agent/opa/releases/download/v${OPA_VERSION}/
opa_linux_amd64_static" \
            -o /usr/local/bin/opa
          chmod +x /usr/local/bin/opa
          conftest --version && opa version

      - name: Install Trivy
        run: |
```

```
          set -euo pipefail
          curl -fsSL "https://github.com/aquasecurity/trivy/releases/download/v${TRIVY_VERSION}/
trivy_${TRIVY_VERSION}_Linux-64bit.tar.gz" \
            | tar -xz -C /usr/local/bin trivy
          trivy --version

      # The gate's own tests run before it is trusted to gate anything.
      - name: opa test
        run: opa test -v policies/

      # An unconstrained policy enforces nothing. Prove each one can fail.
      - name: Mutation-test the policy suite
        run: bash scripts/mutation-test.sh policies

      - name: terraform fmt
        run: terraform fmt -check -recursive terraform/

      - name: Terraform plan
        working-directory: ${{ env.TF_WORKING_DIR }}
        run: |
          set -euo pipefail
          # An unset repository variable renders as an EMPTY string, not as a
          # missing one, and Terraform accepts empty and builds from it. These
          # two feed the bucket names, so empty means a plan that proposes
          # renaming everything you already have. Fail here instead.
          : "${TF_VAR_project_name:?set the TF_VAR_PROJECT_NAME repository variable}"
          : "${TF_VAR_environment:?set the TF_VAR_ENVIRONMENT repository variable}"
          # The backend is partial on purpose: main.tf carries the key, and
          # backend.hcl carries the bucket and is gitignored because it names
          # your account's resources. CI therefore has no backend.hcl, and a
          # bare `terraform init -input=false` fails outright rather than
          # prompting. The bucket arrives as a repository variable instead,
          # which is what `gh variable set TF_STATE_BUCKET` in Step 2 is for.
          terraform init -input=false \
            -backend-config="bucket=${{ vars.TF_STATE_BUCKET }}"
          terraform validate
          terraform plan -out=tfplan -no-color | tee plan.txt
          terraform show -json tfplan > plan.json

      - name: Conftest policy gate
        id: conftest
        run: |
          mkdir -p evidence
          bash scripts/policy-gate.sh \
            --workspace "${TF_WORKING_DIR}" \
            --policy policies \
            --evidence evidence

      - name: Trivy config scan
        id: trivy
        if: always()
        run: |
          set -euo pipefail
          # --ignorefile is required: Trivy auto-discovers a plain
          # .trivyignore but NOT .trivyignore.yaml, and the YAML form is the
          # one that carries a justification and an expiry date.
```

```
          trivy config "${TF_WORKING_DIR}" \
            --ignorefile .trivyignore.yaml \
            --format sarif --output evidence/trivy.sarif \
            --severity HIGH,CRITICAL --exit-code 0
          COUNT=$(jq '[.runs[].results[]] | length' evidence/trivy.sarif)
          echo "trivy HIGH+CRITICAL: $COUNT"
          jq -r '.runs[].results[] | "  " + .ruleId + ": " + .message.text' evidence/trivy.sarif ||
true
          test "$COUNT" -eq 0

      # if: always() on everything below. A Conftest failure must not abort
      # the job before evidence is captured; a failed run is exactly the run
      # whose evidence you will want later.
      - name: Copy plan into evidence
        if: always()
        run: |
          cp "${TF_WORKING_DIR}/plan.json" evidence/plan.json || true
          cp "${TF_WORKING_DIR}/plan.txt"  evidence/plan.txt  || true

      - name: Upload evidence artifact
        if: always()
        uses: actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a # v7.0.1
        with:
          name: grc-evidence-${{ github.run_id }}
          # Named files, not the evidence/ directory.
          #
          # `path: evidence/` uploads whatever else lives there, which in a real
          # repository is every prior lab's artifacts, including
          # evidence/lab-2-3/state.json. Terraform state records every attribute
          # the provider stored, sensitive ones included, and Lab 2.5 exists to
          # teach exactly that. Re-uploading it on every run, to an artifact any
          # repository reader can download, is the opposite of the lesson.
          #
          # It also nests: download a run's artifact into evidence/ and the next
          # run packages it inside the next artifact, and so on.
          path: |
            evidence/plan.json
            evidence/plan.txt
            evidence/conftest-results.json
            evidence/trivy.sarif
          # 90, not 365, and the difference is a control claim.
          #
          # GitHub caps artifact retention at the repository or organisation
          # maximum, 90 days by default. Ask for more and it does not fail: it
          # warns, silently stores 90, and the run still goes green. A pipeline
          # configured for 365 that actually retains 90 is a documented AU-11
          # retention period your evidence does not have.
          #
          # If you need longer, raise the limit in repository settings first,
          # then change this, then confirm the actual retention on a real
          # artifact. Do not cite a retention period you have not checked.
          retention-days: 90
```

Choices worth understanding:

- `permissions: id-token: write` is required for OIDC. Without it the credential step fails in a way that looks like an AWS problem and is not.

- `if: always()` on the scan, evidence copy, and upload. Without it a Conftest failure aborts the job before evidence is captured, and the entire point of CI evidence is that it survives the failure.
- **Actions pinned by SHA.** "Pin your actions" is common advice usually illustrated with mutable tags, which misses the point. `@v4` is a pointer the upstream maintainer can move, and has moved, including during at least one widely-reported supply chain incident. The comment after the SHA keeps it readable.

> **A pin is a maintenance obligation, not a one-time decision.** This is the half of the story most guidance leaves out, and it is worth internalizing before you adopt the practice.
>
> A tag floats, so it silently picks up fixes and silently picks up compromises. A SHA is frozen, so it does neither. What it also does is stop receiving the upstream's runtime migrations, and eventually you are running something the platform is deprecating under you.
>
> This curriculum's own CI demonstrated it. Every job passed, and every job carried:
>
> ```
> Node.js 20 is deprecated. The following actions target Node.js 20 but are
> being forced to run on Node.js 24: actions/checkout@11bd719...
> ```
>
> Nothing was broken. The pins were simply old enough that GitHub was force-migrating them off their target runtime, and the only signal was a warning in a green run that nobody reads.
>
> So pin, and pair it with a refresh process: Dependabot or Renovate raising a PR per bump, a quarterly review, or at minimum a CI job that fails when a pinned action is more than N releases behind. **A pin with no refresh process converts a supply chain risk into an obsolescence risk.** You have not removed the problem, you have changed which problem you have, and the second one is quieter.
>
> Trading one risk for a quieter one is still usually the right trade. Just make it knowingly, and write down who checks.

- `retention-days: 365`, not the 90-day default. Lab 4.4 mirrors the bundle into the vault, but the artifact should outlive the audit cycle regardless.
- **Policy tests run before the gate.** A suite that fails its own unit tests should not be trusted to decide whether your infrastructure ships.

Step 4 Trivy, not tfsec

Much older guidance reaches for `tfsec`. It has been folded into Trivy by its maintainer, Aqua Security, and the misconfiguration checks now live there. `trivy config` runs the same rule set, is actively maintained, and covers Terraform, CloudFormation, Kubernetes, and Dockerfiles with one binary.

Point it at your own Lab 2.3 workspace before you point it at the starter. Trivy flags missing HTTPS enforcement on a bucket policy and SSE-S3 instead of a customer-managed key, both at HIGH. Lab 2.3 closes both, so the gate stays quiet. Take either shortcut and this gate, configured to fail closed on HIGH, will fail your own Chapter 2 artifact.

That agreement between your baseline and your gate is not automatic. It is the check to run whenever you change the baseline.

Step 5 Open a PR and watch it run

```
git checkout -b add-grc-gate
git add .github/workflows/grc-gate.yml policies/ scripts/ oidc/ terraform/
git commit -m "Add GRC evidence pipeline"
git push -u origin add-grc-gate
gh pr create --title "Add GRC evidence pipeline" --body "Reference pipeline."
gh run watch
```

Against the compliant Lab 2.3 workspace:

```
policy-gate: PASS
trivy HIGH+CRITICAL: 0
```

Against the `cgep-app-starter`, which ships eight intentional gaps, both gates fire. That is the expected outcome and the starting point for the capstone.

Step 6 The two-PR demonstration

The capstone wants a green PR and a red PR in your history.

1. **Red**: branch that introduces a violation. Delete `aws_s3_bucket_policy.primary` (SC-8), or set `block_public_acls = false` (AC-3), or drop `aws_s3_bucket_versioning.log` (AU-9). Open the PR. The gate fails, the merge is blocked.
2. **Green**: fix it. The gate passes, the PR merges.

Both runs leave evidence artifacts. Both URLs go in your write-up.

Turn on the branch protection that makes this mean something:

```
gh api -X PUT "repos/YourOrg/YourRepo/branches/main/protection" \
  --input - <<'EOF'
{
  "required_status_checks": { "strict": true, "contexts": ["grc-gate"] },
  "enforce_admins": true,
  "required_pull_request_reviews": { "required_approving_review_count": 1 },
  "restrictions": null
}
EOF
```

Without branch protection the gate is advisory. A red check that anyone can merge past is not CM-3; it is a suggestion. `enforce_admins: true` is what makes it a control rather than a habit, and it is the first thing an assessor will check.

Step 7 The inert-gate test

Before you trust this, prove it can fail for the right reason:

```
git checkout -b test-inert-gate
git rm -r policies/
git commit -m "TEST: remove policies (expect gate failure, do not merge)"
git push -u origin test-inert-gate
gh pr create --title "TEST: inert gate" --body "Expect failure."
```

The run must go **red**, with `policy-gate: FAIL (no policy results produced; gate did not run)`. If it goes green, your gate reports success when it is evaluating nothing, which is the worst possible failure mode: a confident dashboard over an unexamined system.

Close the PR without merging and keep the run URL. It is the best single piece of evidence in your portfolio that the gate is real.

Takeaway: the YAML is the control statement

Workflow content	Control
`on: pull_request` + branch protection requiring this check	CM-3 (configuration change control)
`terraform fmt -check` and the Conftest tag rule	CM-6 (configuration settings)
The workflow as a recurring assessment	CA-2, CA-7
Trivy scanning every change	RA-5
Actions pinned by SHA, tools by version	SR-3, SR-11 (supply chain)
OIDC instead of stored access keys	IA-5, AC-6
Evidence artifacts retained 365 days, signed in Lab 4.4	AU-9

The workflow file is checked in, the history is preserved, and an assessor traversing your OSCAL component in Lab 6.1 follows an evidence URI that lands on this workflow's output.

Verification

- A PR triggers the workflow and it appears in the Actions tab.
- `grc-evidence-<run-id>` is attached with `plan.json`, `plan.txt`, `conftest-results.json`, `trivy.sarif`.
- Compliant code: green. Non-compliant: red, with control IDs in the log.
- Removing `policies/` makes it red, not green.
- Branch protection blocks merge on red, including for admins.

Upload named files, never the evidence directory. The artifact is called `grc-evidence-<run-id>`, so it should hold what that run produced: the plan, the plan text, the Conftest results, the Trivy SARIF. Pointing `upload-artifact` at `evidence/` instead sweeps in everything else that lives there, which in a real repository means every earlier lab, including `evidence/lab-2-3/state.json`.

Terraform state records every attribute the provider stored, sensitive values included. Lab 2.5 is built around that fact. Re-uploading it on every run, to an artifact anyone with read access can download, teaches the opposite.

It also compounds. Download a run's artifact into `evidence/` to keep as evidence, and the next run packages that inside the next artifact, and the one after that packages both.

The artifact retention warning is a control claim, not a nuisance. The workflow used to ask for `retention-days: 365`. GitHub caps artifact retention at the repository maximum, 90 days by default, and asking for more does not fail. It prints a warning, stores 90, and the run goes green.

That is a pipeline documented as retaining evidence for a year that retains it for a quarter. If AU-11 appears in your OSCAL with a 365 day retention period, sourced from this workflow, the claim is false and nothing in the run told you so. It is the same failure this course keeps circling: the system did what you asked as far as it could, said so quietly, and carried on green.

It now asks for 90, which is what it gets. If you need longer, raise the limit in repository settings, change the value, and then confirm the actual retention on a real artifact before you cite it anywhere.

Portfolio submission checklist

- [ ] `oidc/` module committed, with the state-access policy.
- [ ] `.github/workflows/grc-gate.yml` committed, every action SHA-pinned.
- [ ] `.trivyignore.yaml` committed, every entry carrying a justification and an `expired_at`.
- [ ] `vars.AWS_ROLE_ARN` set.
- [ ] One green PR and one red PR in history.
- [ ] The inert-gate test run URL, closed unmerged.
- [ ] Branch protection enabled with `enforce_admins: true`, screenshot or `gh api` output in `evidence/lab-4-3/`.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`Could not assume role with OIDC: invalid identity token`.** The `sub` condition does not match. PR runs use `repo:OWNER/REPO:pull_request`; branch pushes use `repo:OWNER/REPO:ref:refs/heads/BRANCH`. `StringLike` with `repo:OWNER/REPO:*` covers both.
- **`AccessDenied` on `s3:PutObject` during `terraform init`.** The role needs write to the state bucket and `kms:GenerateDataKey` on the state key. `ReadOnlyAccess` alone is not enough; that is what `aws_iam_role_policy.state_access` is for.
- **`Error acquiring the state lock` on concurrent PRs.** Two runs against one state `key`. Serialize with a concurrency group, or give PR runs a separate key.
- **Conftest finds no policies.** `--policy` resolves relative to the step's working directory. The gate script takes it as an argument for exactly this reason.
- **Trivy findings you disagree with.** Put them in `.trivyignore.yaml` at your repository root, with a justification and an `expired_at` date, and never scatter `--skip` flags through the workflow: an exclusion nobody can find is an exclusion nobody reviews. Use the **YAML** file rather than a plain `.trivyignore`, because only the YAML form carries a statement and an expiry, and pass `--ignorefile` explicitly, because Trivy auto-discovers the plain file and not the YAML one.

  This lab ships one, and it is worth reading as the worked example. Trivy reports `AWS-0132`, "bucket does not encrypt data with a customer managed key", against a plan where both buckets name the same CMK. It reads encryption from the `aws_s3_bucket` resource, and since AWS provider v4 that setting lives in a separate `aws_s3_bucket_server_side_encryption_configuration` resource which Trivy does not correlate back in plan mode. The tell is that two identically configured buckets produce one finding rather than two.

  Notice what makes the suppression defensible rather than convenient. The reason is written down, a command to re-check the reason is written down, `expired_at` makes Trivy surface the finding again on its own, and the control is enforced anyway by `policies/sc28_encryption_aws.rego`, which asserts the same property from the same plan and is mutation-tested. Suppressing a scanner rule you have independently enforced is an exception. Suppressing one you have not is a hole with a comment on it.
- **The plan shows every resource as new.** Your backend block is missing or points at the wrong key. See the prerequisite note at the top.

Cleanup

Delete test branches. The workflow file stays; it is the deliverable. The IAM role and OIDC provider stay; they are free.

How this feeds the capstone

This is the capstone's pipeline. Lab 4.4 adds Cosign signing and the vault upload; the capstone adds the apply step.

For that apply step, use a separate job gated on `github.ref == 'refs/heads/main'`, a GitHub environment with a protection rule, and a **second** IAM role whose trust policy binds to `repo:OWNER/REPO:environment:production`. The plan role stays read-only. Splitting plan credentials from apply credentials is the single highest-value thing you can say in your write-up about this layer, and it is only possible because Lab 2.2 gave you somewhere for the two jobs to share state.

Revision history

These notes

- Requires the Lab 2.2 remote state backend. Without it a fresh runner plans against empty state, which makes the capstone's apply-on-merge step uncompletable.
- The OIDC role gained an explicit state-access policy alongside `ReadOnlyAccess`, because a plan writes a lock object and refreshes state.
- `tfsec` replaced by `trivy config`, its maintained successor.
- Actions pinned by commit SHA rather than by tag, with a note that a pin is a maintenance obligation.
- Added policy unit tests and the mutation test as pipeline steps, ahead of the gate that depends on them.
- Added the inert-gate test: removing `policies/` must turn the run red.
- Branch protection with `enforce_admins: true` documented as the thing that makes the gate a control rather than a suggestion.
- Evidence artifact retention raised from 90 to 365 days.

The official labs

Initial release: local state, `tfsec`, actions pinned by tag, no mutation test and no inert-gate test.